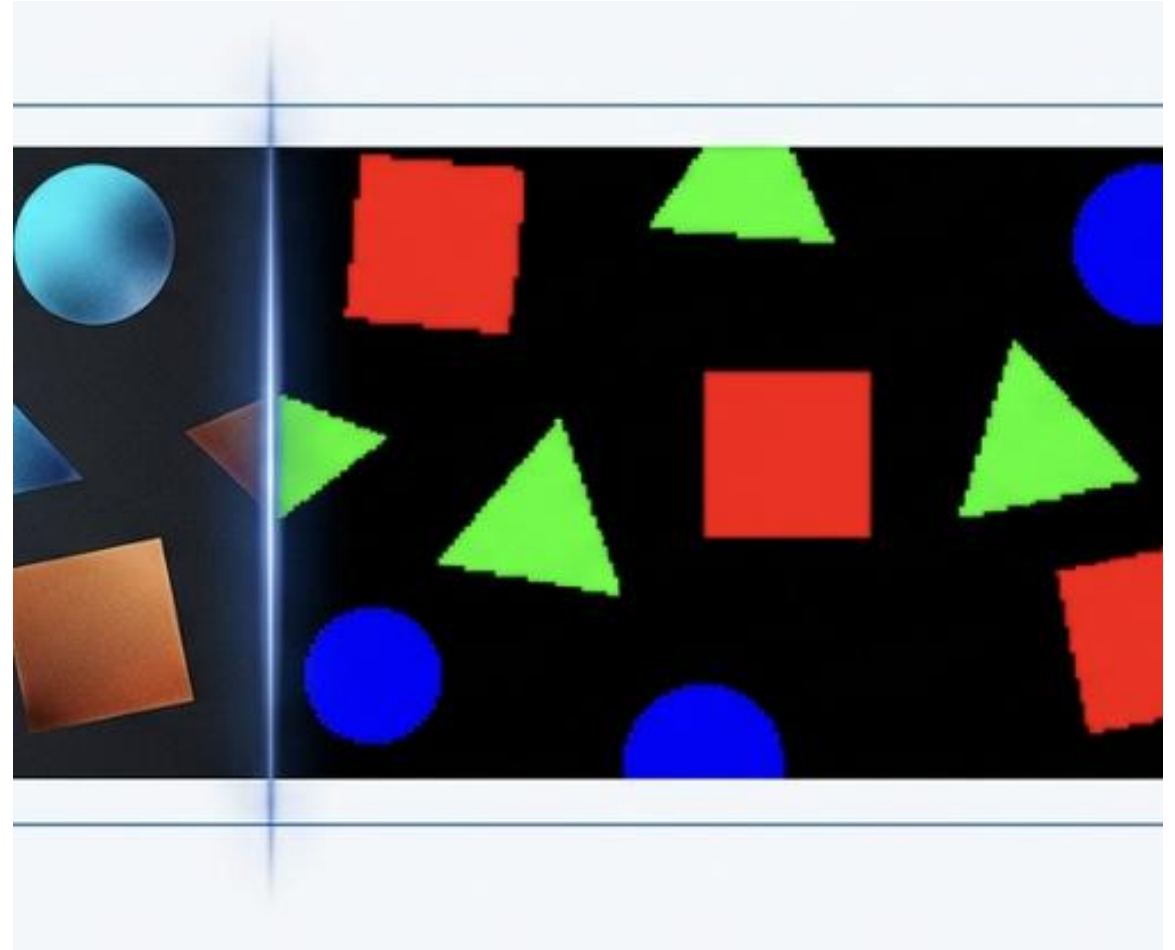


Image Segmentation using CNNs

Student Name : Dorsa Norouzi

Agenda:

- Problem definition and task understanding
- Dataset description and observations
- Network architecture
- Training strategy
- Experimental results and evaluation metrics
- Comparison with and without data augmentation
- Conclusions



Task: Shape-Base Semantic Segmentation

The goal of this project is to perform semantic segmentation on The flying object dataset synthetic images containing simple geometric objects and to assign a semantic class to each pixel of an input image.



- class 1  Red → Square
- class 2  Green → Triangle
- class 3  Blue → Circular
- class 0  Black → Background

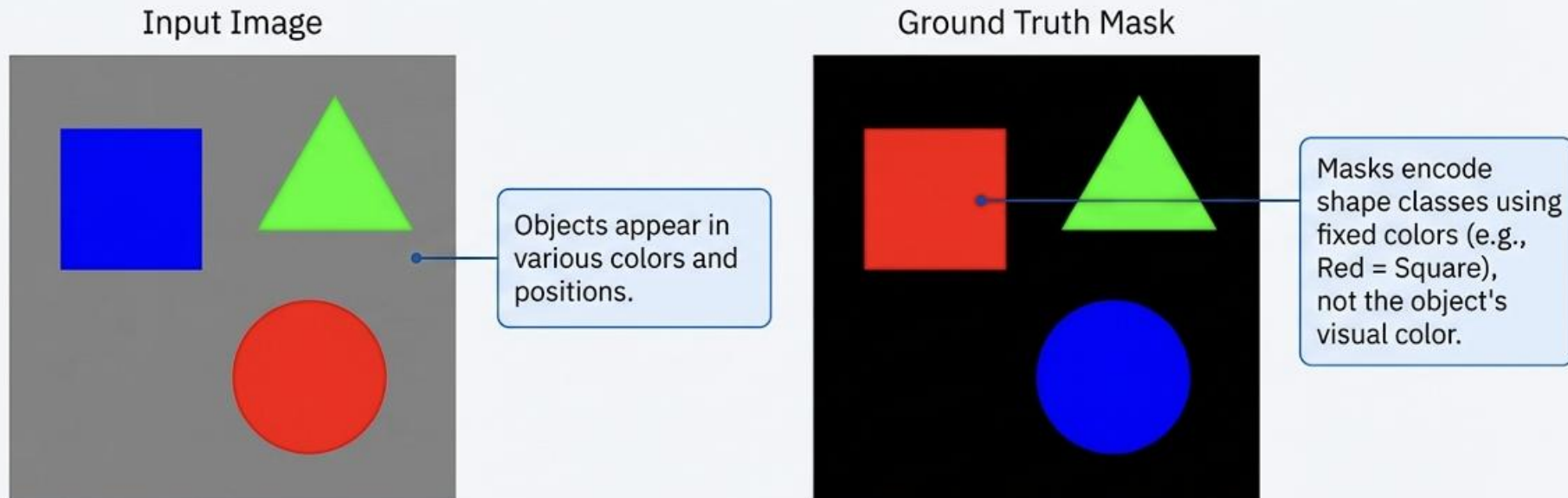
The segmentation is shape-based, not color-based.
Masks encode only shape information
They use fixed color code to present classes

Dataset Overview:

The dataset consists of:

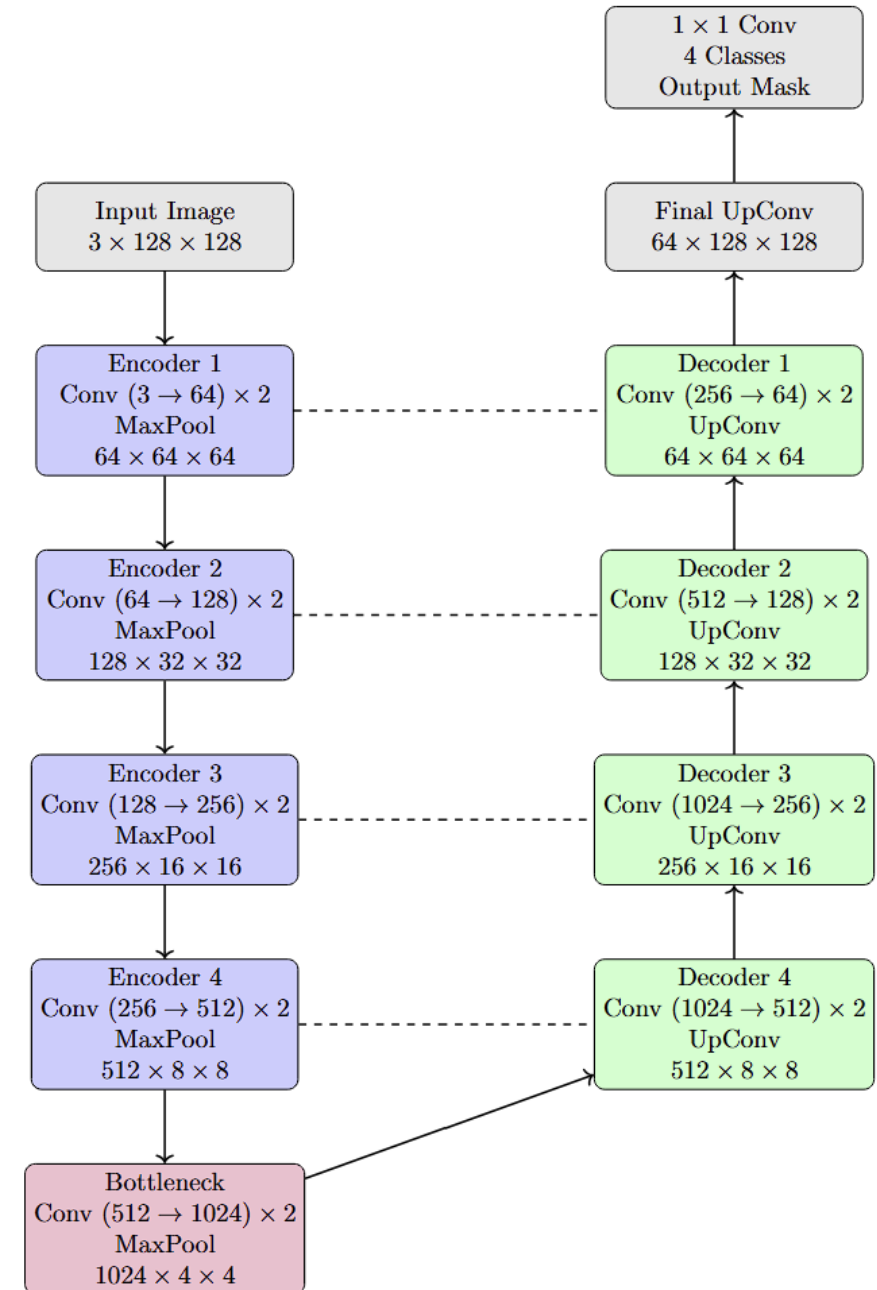
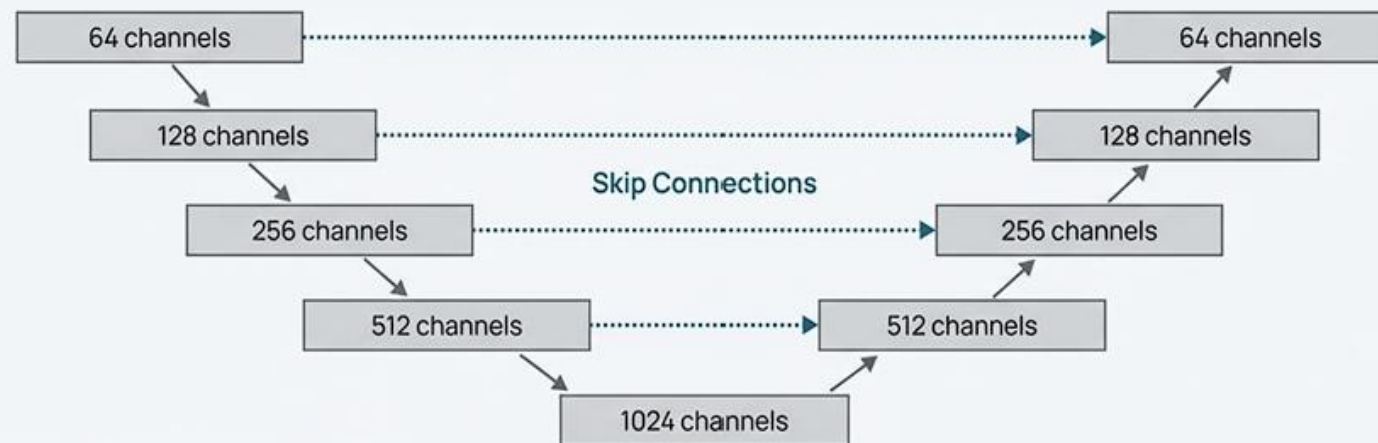
- Input images (image/): RGB images with multiple shapes
- Ground truth masks (gt_image/): Pixel-wise labeled segmentation masks

Important observation:



Network Architecture:

- U-Net–style encoder–decoder architecture
- Four encoder blocks + bottleneck + four decoder blocks
- Two convolution layers per block for increased representational capacity
- Max pooling for downsampling
- Transposed convolutions for upsampling
- Skip connections to preserve spatial structure
- 1×1 convolution for pixel-wise classification



Training Procedure:

Loss Function

Choice: `nn.CrossEntropyLoss()`

Why: The standard and most effective choice for multi-class semantic segmentation, as it computes loss on a per-pixel basis.

Optimizer

Choice: `torch.optim.Adam`

Why: Chosen for its fast convergence and stable training performance on CNNs.

Hyperparameters

Learning Rate: `1e-3`

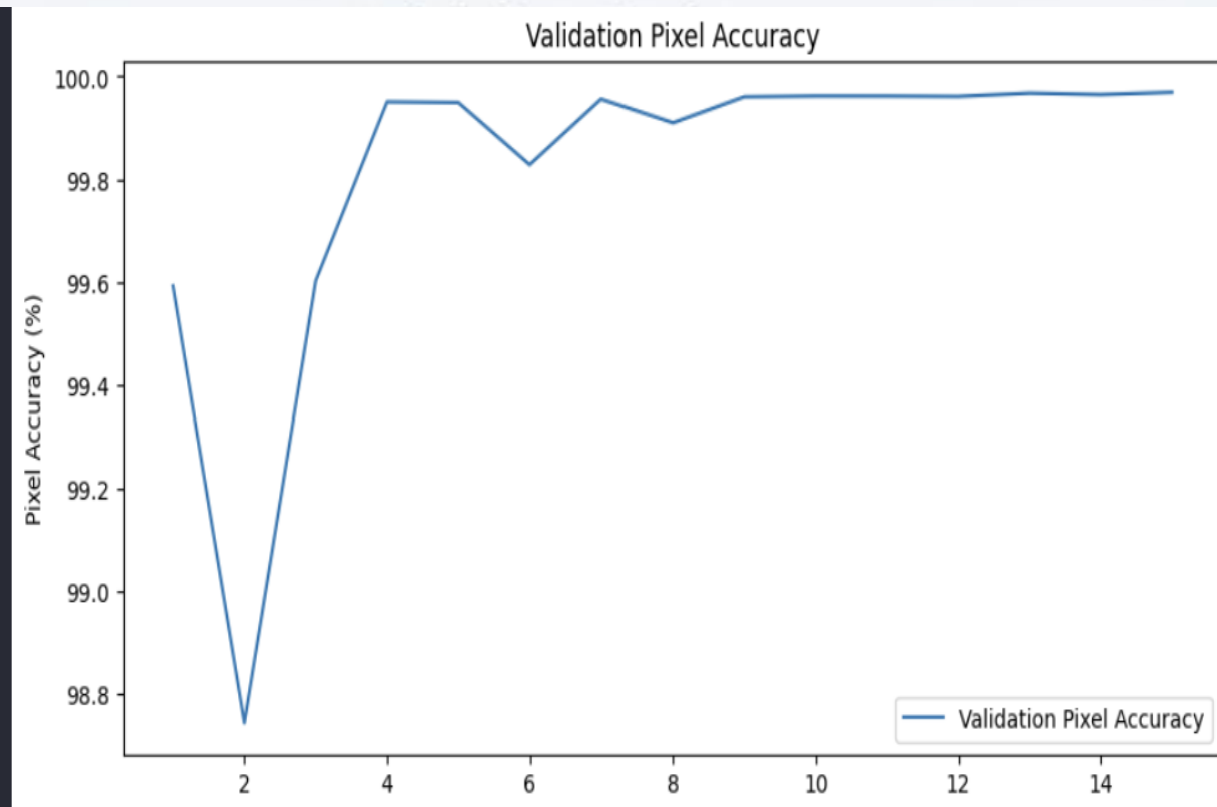
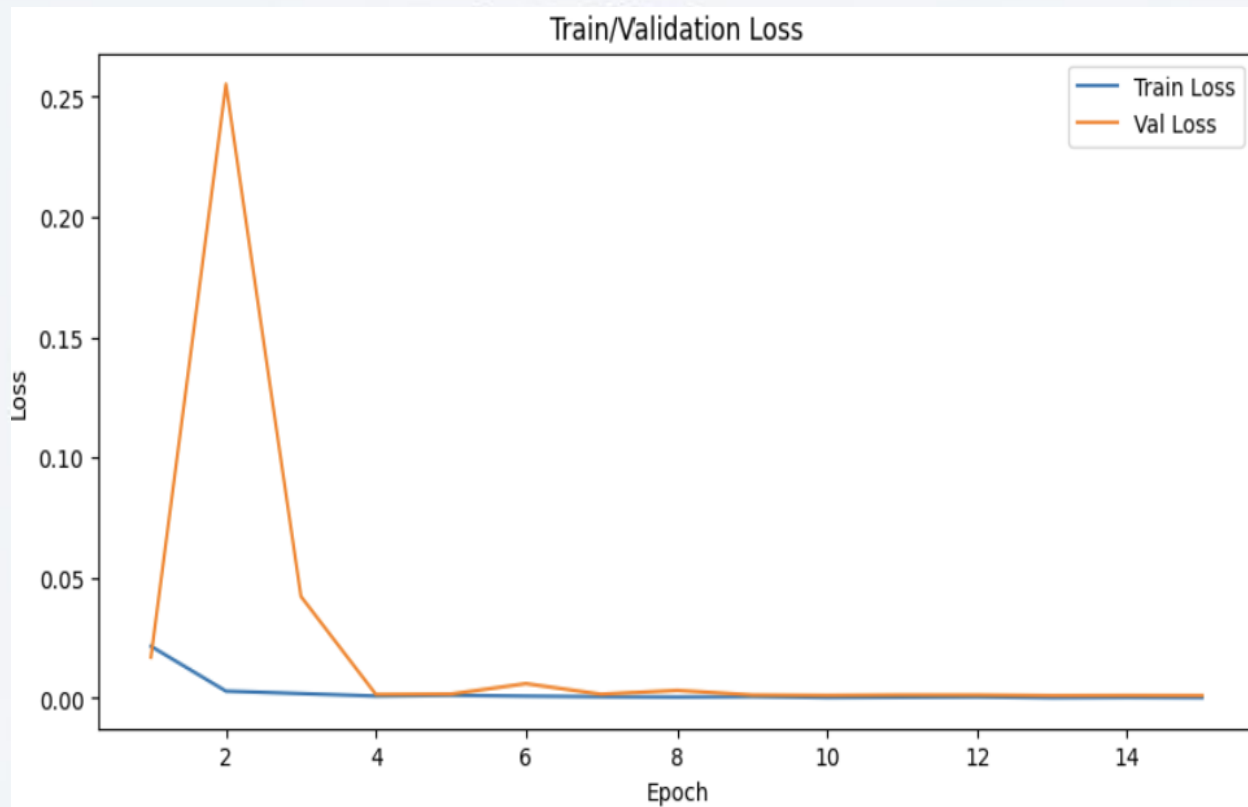
Batch Size: `4`

Epochs: `15`

Initial Training Shows Near-Perfect Performance

After training for 15 epochs, the model achieves an impressive result on the test set.

99.96% Test Pixel Accuracy



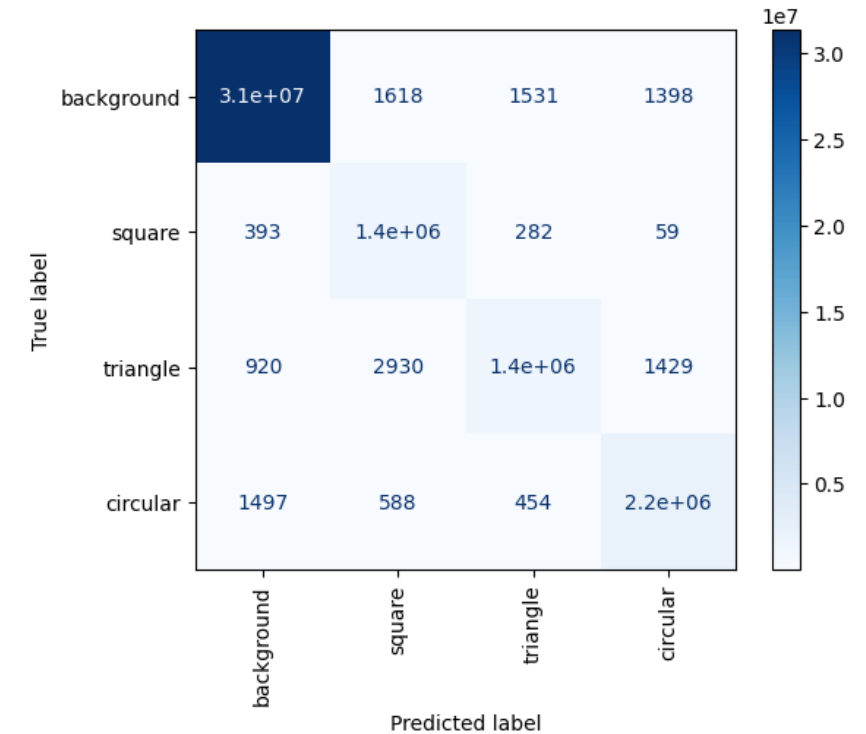
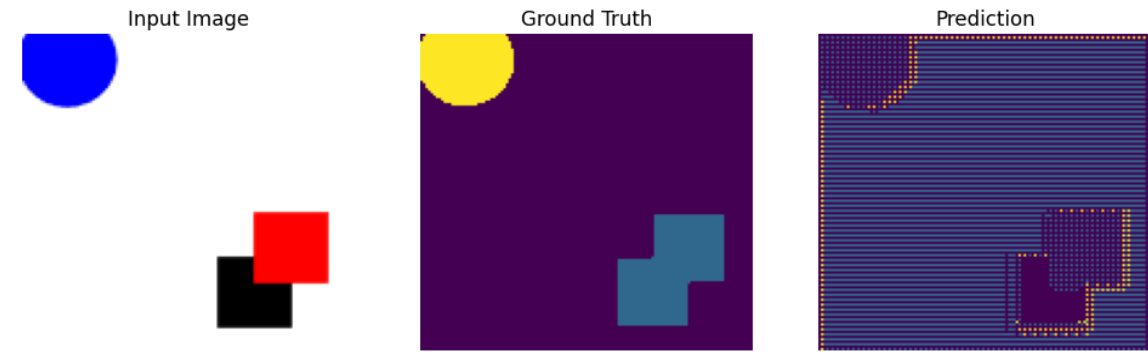
Result accuracy:

Table1: Per-class segmentation performance on the test set

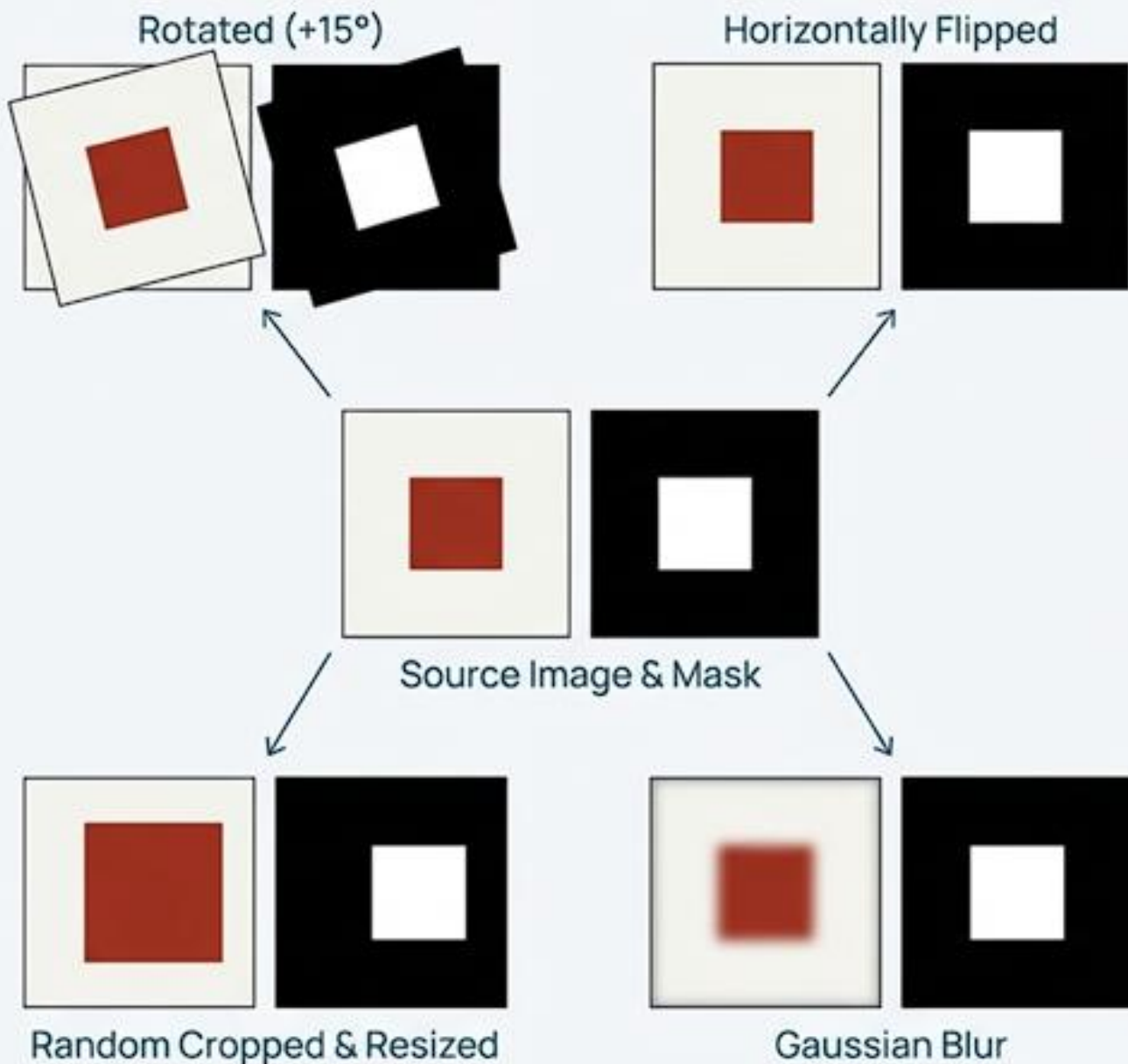
Class	Precision	Recall	F1-score	IoU	Dice	Support
Background	0.9999	0.9999	0.9999	0.9998	0.9999	31,418,517
Square	0.9962	0.9995	0.9978	0.9957	0.9978	1,351,993
Triangle	0.9984	0.9963	0.9974	0.9947	0.9974	1,427,563
Circular	0.9987	0.9988	0.9988	0.9975	0.9988	2,174,40

Table2: Overall segmentation Performance metrics on the dataset

Metric	Value
Pixel-wise Accuracy	99.96%
Mean IoU (mIoU)	0.9969
Mean Dice	0.9985
Macro Avg F1-score	0.9985
Weighted Avg F1-score	0.9996



Augmentation in Action with Albumentations



Code Implementation

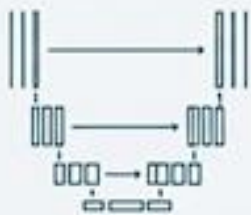
```
import albumentations as A

train_transform_aug = A.Compose([
    A.RandomResizedCrop(height=128,
                        width=128, scale=(0.8, 1.0)),
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.Rotate(limit=30, p=0.5),
    A.GaussianBlur(blur_limit=3, p=0.3),
])
```

Comparison with and without data augmentation:

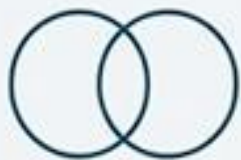
Metric	Without Augmentation	With Augmentation
Test Pixel Accuracy	99.96%	99.73%
Mean IoU	0.9969	0.9833
Mean Dice	0.9985	0.9915
Per-class IoU	Slightly higher on all classes	Slightly lower but more robust
Per-class F1	Very high	Very high

Key Lessons from the Project



1. **U-Net is highly effective for segmentation.**

Its skip connections are vital for preserving the spatial detail necessary for precise pixel-level tasks.



2. **Pixel accuracy is insufficient for segmentation.**

It can be misleading due to class imbalance. IoU and Dice provide a far more reliable measure of performance.



3. **Data augmentation is critical for building robust models.**

A slight decrease in metrics on a clean test set is a worthwhile trade-off for a model that generalizes better to real-world variations.



4. **The final augmented model achieves near-perfect segmentation,**

a result validated by a comprehensive suite of both quantitative and qualitative evidence.